南開大學
Nankai University

数据结构实验报告八

AVL 树

学号：2412266

姓名： 杨滢

专业：信息安全

学院：密码与网络空间安全学院

目录

1 题目一

1.1 题目表述

编写函数，计算 AVL 树的高度，要求说明该函数是所有算法中最优的。

1.2 代码 1 的测试用例

约定空树高度为 0，非空树高度为根节点到叶子节点的最长路径上的节点数，f 为编写的函数，，T 为树。

1. f(T) → 0 # 空树
2. insert(T, 30), f(T) → 1
3. insert(T, 40), insert(T, 50), f(T)→ 2
4. insert(T, 20), insert(T, 10), f(T) → 3
5. insert(T, 25), f(T) → 3
6. insert(T, 60), insert(T, 70), f(T)→ 4
7. erase(T, 10), erase(T, 25), f(T) → 3

1.3 思路

1.3.1 高度查询

插入和删除操作之后都进行高度更新，非空节点高度 = 1 + max(左子树高度, 右子树高度)。

```
void updateHeight(Node* node) {
    if (node != nullptr) {
        node->height = 1 + max(getTreeHeight(node->left),
            getTreeHeight(node->right));
    }
}
```

使用 f(T) 函数直接返回根节点存储的高度值

```
int getTreeHeight(Node* root) {
    if (root == nullptr) {
        return 0;
    }
    return root->height;
}
int f(Node* T) {
    return getTreeHeight(T);
}
```

1.3.2 AVL 树平衡

1. 高度管理:

   getHeight() 函数安全地获取节点高度，处理空节点情况。每次插入或删除后更新相关节点的高度。

2. 四种旋转情况

   - 左左情况 (LL)：对失衡节点右旋转
   - 右右情况 (RR)：对失衡节点左旋转
   - 左右情况 (LR)：先左旋左子树，再右旋当前节点
   - 右左情况 (RL)：先右旋右子树，再左旋当前节点

1.4 代码

```
#include <iostream>
#include <fstream>
#include <algorithm>
#include <string>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
    int height;
    Node(int val) {
        data = val;
        left = right = nullptr;
        height = 1;
    }
};
int getTreeHeight(Node* root) {
    if (root == nullptr) {
        return 0;
    }
    return root->height;
}
void updateHeight(Node* node) {
    if (node != nullptr) {
        node->height = 1 + max(getTreeHeight(node->left),
            getTreeHeight(node->right));
    }
}
```

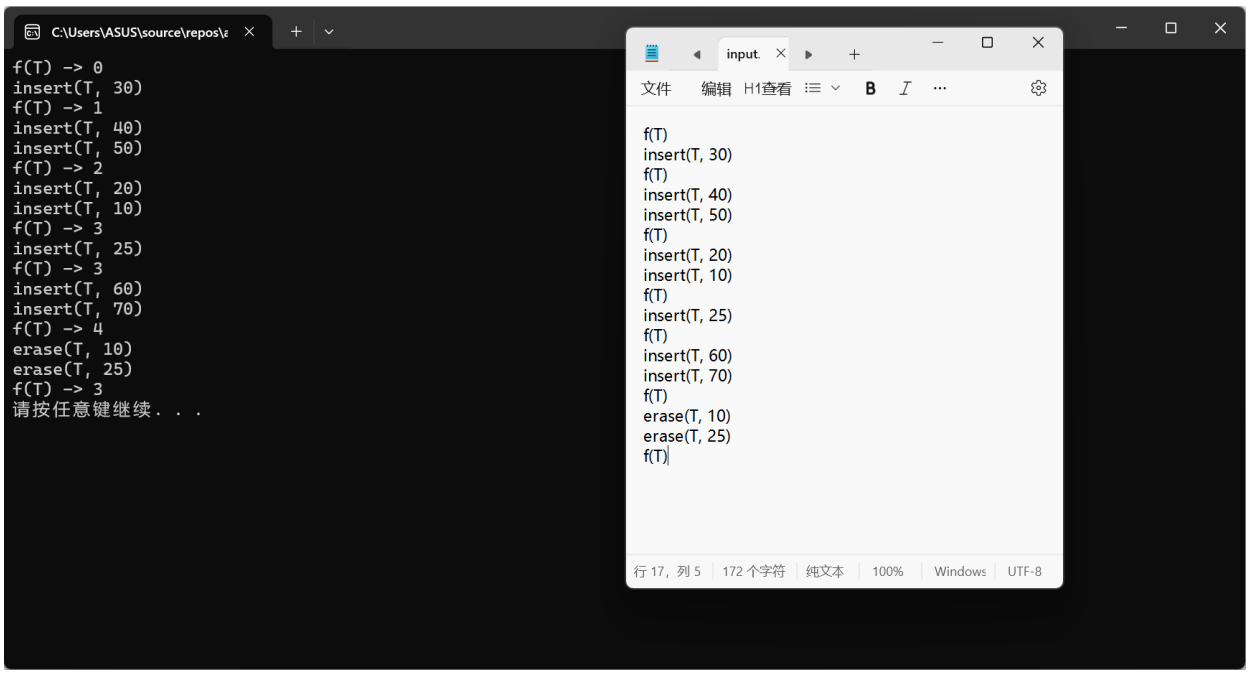
```
Node* rotateRight(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    updateHeight(y);
    updateHeight(x);
    return x;
}
Node* rotateLeft(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;
    y->left = x;
    x->right = T2;
    updateHeight(x);
    updateHeight(y);
    return y;
}
int getBalance(Node* node) {
    if (node == nullptr)
        return 0;
    return getTreeHeight(node->left) - getTreeHeight(node->right);
}
Node* insertNode(Node* node, int data) {
    if (node == nullptr) {
        return new Node(data);
    }
    if (data < node->data) {
        node->left = insertNode(node->left, data);
    }
    else if (data > node->data) {
        node->right = insertNode(node->right, data);
    }
    else {
        return node;
    }
    updateHeight(node);
    int balance = getBalance(node);
    if (balance > 1 && data < node->left->data) {
        return rotateRight(node);
    }
    if (balance < -1 && data > node->right->data) {
```

```
70          return rotateLeft(node);
71      }
72      if (balance > 1 && data > node->left->data) {
73          node->left = rotateLeft(node->left);
74          return rotateRight(node);
75      }
76      if (balance < -1 && data < node->right->data) {
77          node->right = rotateRight(node->right);
78          return rotateLeft(node);
79      }
80      return node;
81  }
82
83  Node* findMinNode(Node* node) {
84      Node* current = node;
85      while (current->left != nullptr) {
86          current = current->left;
87      }
88      return current;
89  }
90
91  Node* deleteNode(Node* root, int data) {
92      if (root == nullptr) return root;
93
94      if (data < root->data) {
95          root->left = deleteNode(root->left, data);
96      }
97      else if (data > root->data) {
98          root->right = deleteNode(root->right, data);
99      }
100     else {
101         if (root->left == nullptr || root->right == nullptr) {
102             Node* temp = root->left ? root->left : root->right;
103             if (temp == nullptr) {
104                 temp = root;
105                 root = nullptr;
106             }
107             else {
108                 *root = *temp;
109             }
110             delete temp;
111         }
```

```
        else {
            Node* temp = findMinNode(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }
    if (root == nullptr) return root;
    updateHeight(root);
    int balance = getBalance(root);
    if (balance > 1 && getBalance(root->left) >= 0) {
        return rotateRight(root);
    }
    else if (balance > 1 && getBalance(root->left) < 0) {
        root->left = rotateLeft(root->left);
        return rotateRight(root);
    }
    else if (balance < -1 && getBalance(root->right) <= 0) {
        return rotateLeft(root);
    }
    else if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rotateRight(root->right);
        return rotateLeft(root);
    }
    else
        return root;
}
int f(Node* T) {
    return getTreeHeight(T);
}
int main() {
    Node* myTree = nullptr;
    ifstream inputFile("input.txt");
    if (!inputFile) {
        cout << "WRONG" << endl;
        system("pause");
        return 1;
    }
    string line;
    while (getline(inputFile, line)) {
        line.erase(remove(line.begin(), line.end(), ' '), line.end());
        if (line == "f(T)") {
            cout << "f(T) -> " << f(myTree) << endl;
```

```
        }
        else if (line.find("insert(T,") != string::npos) {
            size_t start = line.find("insert(T,") + 9;
            size_t end = line.find(")", start);
            string valueStr = line.substr(start, end - start);
            int value = stoi(valueStr);
            myTree = insertNode(myTree, value);
            cout << "insert(T, " << value << ")" << endl;
        }
        else if (line.find("erase(T,") != string::npos) {
            size_t start = line.find("erase(T,") + 8;
            size_t end = line.find(")", start);
            string valueStr = line.substr(start, end - start);
            int value = stoi(valueStr);
            myTree = deleteNode(myTree, value);
            cout << "erase(T, " << value << ")" << endl;
        }
    }
    inputFile.close();
    system("pause");
    return 0;
}
```

1.5 测试用例截图



1.6 最优算法

考虑最坏情况：完全不平衡的树，计算 AVL 树高度 n 个节点都需要遍历。

1.6.1 时间复杂度：O(1)

f(T) 函数调用 getTreeHeight(T) 函数，直接检查 nullptr：

- 空树：1 次比较
- 非空树：1 次比较 + 1 次内存访问（读取 height 字段）

时间复杂度恒为 O(1)

1.6.2 空间复杂度：O(1)

```
int f(Node* T) {
    return getTreeHeight(T); // 无递归，立即返回
}
```

- 参数传递：1 个指针
- 返回地址：1 个

达到理论下界，是最优算法。

2 题目二

2.1 题目表述

编写函数，返回 AVL 树中距离根节点最近的叶节点的值。

2.2 代码 2 的测试用例

1. f(T)→ 0
2. insert(T, 30), f(T) → 30
3. insert(T, 40), insert(T, 50), f(T) → 30
4. insert(T, 20), insert(T, 10), f(T) → 50
5. insert(T, 25), f(T) → 10
6. insert(T, 60), insert(T, 70), f(T) → 10
7. erase(T, 10), erase(T, 25), f(T) → 20

2.3 思路

2.3.1 最近叶节点查找

这里使用**广度优先搜索 (BFS)**。
从根节点开始按层次遍历树，返回第一个遇到的叶节点，即最近的叶节点。

2.3.2 插入操作

1. 按 BST 规则递归找到插入位置
2. 创建新节点并链接到树中
3. 回溯更新路径上所有节点的高度
4. 检查并修复平衡性

2.3.3 删除操作

1. 按 BST 规则递归找到要删除的节点
2. 处理三种删除情况：
   - 无子节点：直接删除
   - 一个子节点：用子节点替代
   - 两个子节点：用右子树最小值替代
3. 回溯更新高度并修复平衡

2.4 代码

```
#include <iostream>
#include <fstream>
#include <queue>
#include <algorithm>
#include <sstream>
using namespace std;
struct Node {
    int data;
    Node* left;
    Node* right;
    int height;
};

int getHeight(Node* node) {
    if (node == nullptr)
        return 0;
```

```
    return node->height;
}

Node* createNode(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->left = nullptr;
    newNode->right = nullptr;
    newNode->height = 1;
    return newNode;
}

Node* rightRotate(Node* y) {
    Node* x = y->left;
    Node* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    return x;
}

Node* leftRotate(Node* x) {
    Node* y = x->right;
    Node* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    return y;
}

int getBalance(Node* node) {
    if (node == nullptr) return 0;
    return getHeight(node->left) - getHeight(node->right);
}

Node* insertNode(Node* node, int data) {
    if (node == nullptr)
        return createNode(data);
    if (data < node->data)
        node->left = insertNode(node->left, data);
```

```
59      else if (data > node->data)
60          node->right = insertNode(node->right, data);
61      else
62          return node;
63
64      node->height = 1 + max(getHeight(node->left), getHeight(node->right));
65      int balance = getBalance(node);
66
67      if (balance > 1 && data < node->left->data)
68          return rightRotate(node);
69
70      if (balance < -1 && data > node->right->data)
71          return leftRotate(node);
72
73      if (balance > 1 && data > node->left->data) {
74          node->left = leftRotate(node->left);
75          return rightRotate(node);
76      }
77
78      if (balance < -1 && data < node->right->data) {
79          node->right = rightRotate(node->right);
80          return leftRotate(node);
81      }
82      else
83          return node;
84  }
85
86  Node* findMinNode(Node* node) {
87      Node* current = node;
88      while (current->left != nullptr)
89          current = current->left;
90      return current;
91  }
92
93  Node* deleteNode(Node* root, int data) {
94      if (root == nullptr) return root;
95      if (data < root->data)
96          root->left = deleteNode(root->left, data);
97      else if (data > root->data)
98          root->right = deleteNode(root->right, data);
99      else {
100         if ((root->left == nullptr) || (root->right == nullptr)) {
```

```
            Node* temp = root->left ? root->left : root->right;

            if (temp == nullptr) {
                temp = root;
                root = nullptr;
            }
            else {
                *root = *temp;
            }
            delete temp;
        }
        else {
            Node* temp = findMinNode(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }
    if (root == nullptr) return root;
    root->height = 1 + max(getHeight(root->left), getHeight(root->right));
    int balance = getBalance(root);

    if (balance > 1 && getBalance(root->left) >= 0)
        return rightRotate(root);

    else if (balance > 1 && getBalance(root->left) < 0) {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }

    else if (balance < -1 && getBalance(root->right) <= 0)
        return leftRotate(root);

    else if (balance < -1 && getBalance(root->right) > 0) {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
    else
        return root;
}

int findClosestLeaf(Node* root) {
    if (root == nullptr) return 0;
```

```
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* current = q.front();
        q.pop();
        if (current->left == nullptr && current->right == nullptr) {
            return current->data;
        }
        if (current->left != nullptr) {
            q.push(current->left);
        }
        if (current->right != nullptr) {
            q.push(current->right);
        }
    }
    return 0;
}

int main() {
    Node* tree = nullptr;
    ifstream inputFile("input.txt");
    string line;
    while (getline(inputFile, line)) {
        line.erase(remove(line.begin(), line.end(), ' '), line.end());
        if (line == "f(T)") {
            int result = findClosestLeaf(tree);
            cout << "f(T) -> " << result << endl;
        }
        else if (line.find("insert(T,") != string::npos) {
            size_t start = line.find("insert(T,") + 9;
            size_t end = line.find(")", start);
            string dataStr = line.substr(start, end - start);
            int data = stoi(dataStr);
            tree = insertNode(tree, data);
            cout << "insert(T, " << data << ")" << endl;
        }
        else if (line.find("erase(T,") != string::npos) {
            size_t start = line.find("erase(T,") + 8;
            size_t end = line.find(")", start);
            string dataStr = line.substr(start, end - start);
            int data = stoi(dataStr);
            tree = deleteNode(tree, data);
```

```
185             cout << "erase(T, " << data << ")" << endl;
186         }
187     }
188     inputFile.close();
189     system("pause");
190     return 0;
191 }
```

2.5 测试用例截图